
PracticePro

Failure Analysis & Simplification Roadmap

VEGA Legal Module + Atrium Property Management

React + Convex | Ati Gravity Platform | Nigerian Market

Why This App Might Fail, What to Cut, and How to Ship

May 2026 | Confidential

Table of Contents

1. Executive Summary	4
2. Failure Mode Analysis	4
2.1 Security Failure	5
2.2 Data Integrity Failure	6
2.3 Compliance Failure	6
2.4 Technical Complexity Failure	7
2.5 Market Adoption Failure	8
2.6 Operational Failure	8
3. Complexity Audit: Full Feature Inventory	9
3.1 VEGA Legal Module	9
3.2 Atrium Property Management Module	10
3.3 Cross-Cutting Infrastructure	10
3.4 Complexity Summary	11
4. Simplification Roadmap	11
4.1 KEEP: Essential V1 Features	12
4.2 DEFER: Important But Not Required for V1	12
4.3 CUT: Remove Entirely from Codebase	13
4.4 Recommended V1 Scope	14
5. Audit Framework Mapping to PracticePro	14
5.1 Performance	14
5.2 Security	15
5.3 Accessibility	15
5.4 Code Quality	15
5.5 Data Privacy & Compliance	15
5.6 API Audits	16
5.7 Infrastructure	16
5.8 UX	16

5.9 Database	16
5.10 Compliance Audit Logs	17
6. Priority Action Plan	17
6.1 Phase 1: Security Emergency (Week 1-2)	17
6.2 Phase 2: Core Fixes (Week 3-4)	18
6.3 Phase 3: Simplification (Week 5-6)	18
6.4 Phase 4: Quality & Launch Prep (Week 7-8)	18
7. The Case for Radical Simplicity	19

1. Executive Summary

PracticePro is an ambitious dual-module SaaS application combining VEGA (legal practice management) and Atrium (property management) on the Ati Gravity platform, targeting the Nigerian market. While the vision of an integrated legal-proptech solution is compelling, the current codebase reveals a pattern of accumulated complexity that poses a genuine existential risk to the product. A prior codebase audit identified 5 critical and 8 high-severity issues, and this analysis goes further to examine the systemic reasons why the app, in its current form, is likely to fail at launch.

The core thesis of this report is threefold. First, the app suffers from a compounding complexity problem: each module (legal and property) is itself a full product, and combining them without rigorous abstraction boundaries has created a tangled architecture where failures cascade across modules. Second, critical security and data integrity issues remain unresolved, any one of which could cause a catastrophic breach in a legal context where client confidentiality is a regulatory requirement. Third, the feature scope is far too broad for a V1 launch targeting the Nigerian market, where users need reliability and simplicity over feature completeness.

This report catalogs the specific failure modes, maps the full complexity of the current feature set, and provides a prioritized simplification roadmap that identifies what must be kept, what should be deferred, and what should be cut entirely. The goal is not to diminish the vision but to ensure that PracticePro ships successfully and survives long enough to become the product it was designed to be.

Key Findings at a Glance

Finding	Severity	Impact
Exposed API keys in aiUtils.ts and geminiService.ts	CRITICAL	Financial loss, data breach
withFirmAuth never calls getUserIdentity() - auth bypass	CRITICAL	Any user accesses any firm data
Dual unsynchronized payment systems	CRITICAL	Financial reports unreliable
12-provider context hierarchy	HIGH	Performance, debugging nightmare
Orphaned PropertyTrackingView (dead code)	HIGH	Confusion, wasted dev time
No NBA compliance (audit logs, 7-year retention)	HIGH	Cannot sell to Nigerian law firms
Feature scope too broad for V1	HIGH	Missed deadlines, user confusion

Table 1: Critical findings requiring immediate attention

2. Failure Mode Analysis

This section examines six distinct categories of failure risk. Each is analyzed in terms of the specific codebase issues that create the risk, the likely scenario if the issue is not resolved, and the compounding effects when multiple failure modes interact. Understanding these failure modes is essential for making informed decisions about what to cut and what to keep.

2.1 Security Failure

The most immediate and catastrophic failure mode for PracticePro is a security breach. Legal applications handle some of the most sensitive data in any industry: client communications, case strategies, financial records, and privileged documents. A breach in this context does not merely cause embarrassment or regulatory fines; it can result in disbarment proceedings, civil liability, and the complete destruction of a law firm's reputation. PracticePro currently has three unaddressed security vulnerabilities that, individually and especially in combination, make a security breach not just possible but probable.

Exposed API Keys

The files `aiUtils.ts` and `geminiService.ts` contain hardcoded Gemini API keys. These keys are embedded in the client-side JavaScript bundle, meaning they are accessible to anyone who opens the browser developer tools. An attacker can extract these keys in under thirty seconds and use them to make API calls at PracticePro's expense, potentially running up thousands of dollars in usage charges. More critically, if the Gemini integration has access to firm data (which it does, since AloaChat uses it to answer questions about matters and properties), the attacker may be able to extract confidential client information through carefully crafted prompts. This is not a theoretical risk; it is the single most dangerous vulnerability in the entire codebase and must be fixed before any external user sees the application.

Authentication Bypass via `withFirmAuth`

The Convex authentication wrapper `withFirmAuth` was designed to ensure that only authenticated users can access firm-specific data. However, the prior audit revealed that this wrapper never actually calls `getUserIdentity()`, which is the Convex function that verifies the user's authentication token. This means that the authentication check is essentially a no-op: it goes through the motions of checking firm membership without first verifying that the user is who they claim to be. In practical terms, any unauthenticated HTTP request to a Convex function protected by `withFirmAuth` will succeed, potentially exposing every firm's matters, documents, financial records, and client data. This is an Insecure Direct Object Reference (IDOR) vulnerability, which is consistently ranked in the OWASP Top 10. In a legal application, this is an existential threat.

Compounding Effect

These two vulnerabilities compound each other dangerously. An attacker who extracts the API keys from the client bundle now has a verified entry point into the system. They can use the AI features to probe the data model, learning the schema and relationships. They can then directly query Convex functions protected by the broken `withFirmAuth` to extract data at scale. What makes this particularly insidious is that neither vulnerability triggers obvious alarms: the API usage appears legitimate (it uses real keys), and the data access appears authorized (it passes through the auth wrapper). A breach could persist for weeks or

months before discovery.

2.2 Data Integrity Failure

Data integrity in a legal-proptech application is non-negotiable. Courts, regulatory bodies, and clients expect that financial records are accurate, consistent, and auditable. PracticePro currently has a structural data integrity problem that will cause silent, compounding errors in financial reporting.

Dual Payment Systems

The application maintains two separate and unsynchronized systems for tracking rent payments: `ledger_entries` and `property.rentPaymentHistory`. These two systems can and will diverge over time. When a tenant pays rent, the payment may be recorded in one system but not the other, or recorded with different amounts, different dates, or different statuses. There is no reconciliation mechanism, no sync process, and no single source of truth. This means that at any given moment, the question "How much rent has this tenant paid?" can have two different answers depending on which system you query.

The practical consequences are severe. Landlords relying on PracticePro to manage their properties will make decisions based on incorrect financial data. Tenants may be incorrectly marked as in arrears or, conversely, given credit for payments they never made. In the Nigerian legal context, where property disputes are among the most common types of litigation, inaccurate financial records from a property management system could become evidence in court, and a system that produces contradictory records will be destroyed on cross-examination. Beyond the courtroom, this erodes the fundamental trust that users place in the application. If the numbers cannot be trusted, nothing else matters.

Orphaned Records and Nuclear Delete

The Convex data model lacks proper cascade-delete logic. When a property or a matter is deleted, the associated records (payments, documents, activities) are not automatically cleaned up. Over time, this creates orphaned records that reference non-existent parent entities. These orphans pollute queries, inflate database size, and can cause runtime errors when the UI attempts to render data that references deleted parents. The absence of `firmId` indexes on some tables means that these orphaned records may not even be discoverable through normal queries, making cleanup effectively impossible without a full database scan.

2.3 Compliance Failure

The Nigerian legal market has specific regulatory requirements that PracticePro must meet to be a viable product. The Nigerian Bar Association (NBA) sets professional conduct standards that extend to the tools lawyers use, and any software that handles client data must comply with these standards. Currently, PracticePro fails on three critical compliance dimensions.

Requirement	NBA Standard	Current State	Gap
Audit Logging	Complete record of who accessed what and when	No audit log system exists	Must build from scratch

Data Retention	7-year minimum for client files	No retention policy; Convex default is indefinite	Need policy + enforcement mechanism
Client Confidentiality	Client data isolated between firms	withFirmAuth bypass allows cross-firm access	Critical security fix required
Data Residency	Nigerian data protection regulations	Convex hosts on AWS; region unclear	Need to verify and configure
Consent Management	Client consent for data processing	No consent tracking	Must implement consent flows

Table 2: NBA compliance gap analysis

The most critical gap is the absence of audit logging. The Nigerian Bar Association requires that law firms maintain complete records of who accessed client files, when they accessed them, and what actions they performed. This is not optional; it is a professional conduct requirement. Without an audit log system, PracticePro cannot be sold to any law firm that takes its regulatory obligations seriously. The Convex audit log implementation referenced in the 10-area framework provides a template, but it needs to be adapted for the Nigerian context, including logging compliance-specific events such as matter access, document download, and client data export.

2.4 Technical Complexity Failure

Even if the security and compliance issues were resolved, PracticePro faces a significant risk of failure due to sheer technical complexity. Complexity in software is not just an engineering inconvenience; it is a multiplier on every other risk. Complex systems are harder to test, harder to debug, harder to modify, and harder to onboard new developers to. PracticePro exhibits several patterns of structural complexity that, taken together, create a system that is fragile and expensive to maintain.

12-Provider Context Hierarchy

The application wraps its component tree in twelve separate React context providers. This creates a deeply nested dependency structure where any component in the tree may implicitly depend on context values from any of the twelve providers above it. When a context value changes, all consumers of that context re-render, which in a twelve-provider hierarchy can trigger cascading re-renders across the entire component tree. This degrades performance predictably as the application grows. More critically, it makes debugging extremely difficult: when a bug appears, tracing it through twelve layers of context to find the source requires significant institutional knowledge of the codebase. A new developer joining the team will struggle to understand the data flow, leading to longer development cycles and more bugs introduced by each change.

Modal Context Discard

The ModalManager system, which handles the application's modal dialogs, has a documented issue where context is discarded when modals are opened. In a React application, context values are only available to components that are children of the provider in the component tree. If a modal is rendered outside the provider hierarchy (which is the common pattern for portal-based modals), the modal content cannot access

the context values it needs. This causes modals to render with missing data, broken functionality, or silent failures. The specific instances identified in the prior audit include the CollectRentModal, ReceiptDetailView, and BulkEditPropertyModal. In a property management application, a broken rent collection modal is not just a bug; it is a revenue-blocking defect.

Mega-Query Refactor Risk

The recent Mega-Query refactor was intended to improve performance by consolidating multiple Convex queries into fewer, more efficient queries. While this is a sound optimization strategy, any refactor of data fetching logic carries a high risk of introducing regressions, particularly when the original queries were not covered by automated tests. The refactor may have changed the timing of when data becomes available to components, the order in which related data loads, or the shape of the returned data. Without a comprehensive test suite, these regressions may only surface in production when users encounter edge cases that the developers did not anticipate during manual testing.

2.5 Market Adoption Failure

Even a perfectly engineered product will fail if it does not find product-market fit. PracticePro faces a unique market adoption risk because it is attempting to serve two distinct user personas, lawyers and property managers, with a single integrated product. While there is overlap in the Nigerian market (many lawyers handle property transactions), the core workflows, terminology, and expectations of these two personas are fundamentally different.

A lawyer managing a property dispute does not want to navigate through property management screens to find the matter details. A property manager collecting rent does not want to see legal case management features cluttering their dashboard. The current unified interface tries to serve both personas simultaneously, which means it serves neither well. Each persona sees features that are irrelevant to them, increasing cognitive load and reducing the perceived value of the features that matter to them. In a market where users have low tolerance for complexity and high expectations for simplicity (the Nigerian SaaS market is still maturing, and users often prefer focused tools over Swiss-army-knife platforms), this is a significant barrier to adoption.

Furthermore, the Nigerian legal tech market is still in its early stages. Law firms are transitioning from paper-based and spreadsheet-based systems. The winning products in this market will be those that offer a clear, simple path from the old way of working to the new one. A product that presents lawyers with property management features alongside legal features, many of which are half-built or broken, will confuse and alienate potential early adopters. First impressions are critical in an emerging market, and PracticePro risks squandering its first-mover advantage by presenting an overwhelming interface rather than a focused value proposition.

2.6 Operational Failure

Operational failure is the slowest but most insidious failure mode. It manifests as an ever-increasing cost of development, an ever-decreasing velocity of feature delivery, and an ever-growing backlog of bugs that never seem to get fixed. PracticePro exhibits several structural patterns that predict operational failure.

The dual payment system is a prime example. Every new feature that touches financial data must now account for two different data sources, doubling the testing surface and the likelihood of inconsistencies. The 12-provider context hierarchy means that adding any new feature requires understanding and potentially modifying the provider tree, with each modification risking cascading side effects. The orphaned code (PropertyTrackingView, dead routes) means that developers waste time maintaining code that no user ever sees. Over time, these operational costs compound. Each sprint becomes harder than the last, and the development team finds itself spending more time fighting the architecture than building features.

For a startup targeting the Nigerian market, where resources are constrained and speed to market is critical, operational failure is as deadly as a security breach. A team that cannot ship features quickly will lose to competitors who can, regardless of the quality of the underlying vision. The simplification roadmap in Section 4 is specifically designed to reverse this trend by removing the structural sources of operational drag.

3. Complexity Audit: Full Feature Inventory

This section maps every identified feature across both VEGA and Atrium modules, categorizing each by its current state (working, broken, or orphaned), its complexity contribution, and its importance to the core value proposition. The goal is to create a complete picture of the feature landscape so that informed decisions can be made about what to keep, defer, or cut.

3.1 VEGA Legal Module

Feature	State	Complexity	Core Value	Recommendation
Matter Management (CRUD)	Working	Medium	Essential	KEEP
Document Management	Working	High	Essential	KEEP
Time Tracking / Billing	Working	Medium	Essential	KEEP
Client Management	Working	Medium	Essential	KEEP
Task Management	Working	Low	Important	KEEP
AloaChat (AI Assistant)	Broken (exposed keys)	Very High	Nice-to-have	DEFER
AI Document Analysis	Broken (same keys)	Very High	Nice-to-have	DEFER
Calendar / Deadlines	Partial	Medium	Important	KEEP (simplified)
Legal Research Integration	Not started	High	Nice-to-have	CUT

Court Filing Integration	Not started	Very High	Future	CUT
Custom Workflow Automation	Partial	Very High	Nice-to-have	CUT
Revenue Monitor / Analytics	Partial	High	Enterprise only	DEFER (enterprise)

Table 3: VEGA legal module feature inventory

3.2 Atrium Property Management Module

Feature	State	Complexity	Core Value	Recommendation
Property Listing (CRUD)	Working	Medium	Essential	KEEP
Unit Management	Working	Medium	Essential	KEEP
Tenant Management	Working	Medium	Essential	KEEP
Rent Collection	Broken (modal)	High	Essential	KEEP (fix first)
Receipt Generation	Broken	Medium	Essential	KEEP (fix first)
Payment Tracking	Dual system	Very High	Essential	KEEP (consolidate)
Property Valuations	Broken dialogue	High	Nice-to-have	DEFER
PropertyTrackingView	Orphaned	Low	None	CUT
Bulk Edit Properties	Broken (wrong modal)	Medium	Nice-to-have	DEFER
Maintenance Tracking	Non-functional	Medium	Important	DEFER
Activity Log	Non-functional	Low	Important	DEFER
Add Payment Flow	Non-functional	Medium	Essential	KEEP (fix first)
Automated Receipting	Not working	High	Important	DEFER
Nigerian Tax Calculations	Partial	High	Essential (NG)	KEEP (simplified)
Property Manager vs Owner flows	Unclear separation	High	Important	DEFER (pick one)

Table 4: Atrium property management module feature inventory

3.3 Cross-Cutting Infrastructure

Component	Current State	Issue	Recommendation
Context Provider Tree (12 providers)	Deeply nested	Performance + debugging	Flatten to 4-5 providers
ModalManager System	Context discard	Broken modal features	Redesign with portal context
withFirmAuth Wrapper	Auth bypass	No getUserIdentity() call	Fix immediately
Dual Payment System	Unsynchronized	Data divergence	Consolidate to ledger_entries
DataProvider.tsx	Complex lifecycle	Matter + Property coupling	Separate DataProviders
API Key Management	Hardcoded in client	Exposed to browser	Move to Convex backend
Tier System (tiers.ts)	Centralized	Incomplete enforcement	Keep, enforce at API level
Convex Audit Logging	Not implemented	NBA compliance gap	Build for V1 (minimal)

Table 5: Cross-cutting infrastructure audit

3.4 Complexity Summary

Category	Total Features	Working	Broken / Partial	Orphaned / Not Started
VEGA Legal	12	5	4	3
Atrium Property	15	3	8	4
Cross-Cutting Infra	8	2	6	0
Total	35	10 (29%)	18 (51%)	7 (20%)

Table 6: Feature state distribution

The numbers tell a clear story: only 29% of identified features are in a working state. Over half are broken or partially implemented, and 20% are either orphaned dead code or features that were never started. This is the hallmark of a product that expanded its scope faster than its engineering capacity could support. The path forward requires a disciplined reduction in scope to match engineering capacity, not an increase in engineering capacity to match the scope.

4. Simplification Roadmap

This section provides a concrete, actionable plan for reducing PracticePro's complexity to a level that can be reliably shipped and maintained. Each recommendation is categorized as KEEP (essential for V1 launch), DEFER (important but not required for initial launch), or CUT (remove entirely, either permanently or until

a future major version). The guiding principle is that a simpler product that works reliably will always outperform a complex product that breaks unpredictably.

4.1 KEEP: Essential V1 Features

The features listed below form the minimum viable product for PracticePro. Each one addresses a core user need that cannot be deferred without undermining the product's value proposition. These features must be fully working, tested, and reliable before launch. Any feature in this list that is currently broken must be fixed as a top priority.

Feature	Module	Current State	Action Required
Matter CRUD	VEGA	Working	Stabilize and test
Document Management	VEGA	Working	Add PDF preview
Time Tracking / Billing	VEGA	Working	Connect to billing export
Client Management	VEGA	Working	Verify firm isolation
Property + Unit CRUD	Atrium	Working	Simplify unit model
Tenant Management	Atrium	Working	Link to unit properly
Rent Collection	Atrium	Broken	Fix modal context issue
Receipt Generation	Atrium	Broken	Fix pdfkit + unit-level
Payment Tracking (consolidated)	Atrium	Dual system	Consolidate to ledger_entries
Add Payment Flow	Atrium	Broken	Fix + unit-level context
Nigerian Tax (basic)	Atrium	Partial	Withholding + VAT only
withFirmAuth (fixed)	Infra	Broken	Add getUserIdentity()
API Key Management (server-side)	Infra	Exposed	Move to Convex actions
Basic Audit Logging	Infra	Missing	Login, data access, deletes

Table 7: V1 keep list with required actions

4.2 DEFER: Important But Not Required for V1

These features address real user needs but are not required for the initial launch. They should be revisited in V2 once the core platform is stable and the user base has validated the primary value proposition. Deferring these features reduces the engineering burden by approximately 40% and eliminates the most complex integrations from the V1 scope.

Feature	Reason for Deferral	V2 Priority
AloaChat AI Assistant	Requires API key security fix + prompt engineering + testing; high complexity for uncertain V1 ROI	High
AI Document Analysis	Same API key dependency; legal AI requires domain-specific training data not yet available	Medium
Property Valuations	Broken dialogue; requires data integration with Nigerian property valuation sources	Medium
Bulk Edit Properties	Broken (opens wrong modal); batch operations add complexity without core value	Low
Maintenance Tracking	Non-functional; requires ticketing workflow + notification system	High
Activity Log	Non-functional; design needs alignment with audit log system	High
Automated Receipting	Requires payment gateway integration + email/SMS delivery	High
Revenue Monitor (Enterprise)	Tier-gating incomplete; should only be available for Custom/Enterprise tiers	Medium
Property Manager vs Owner flows	Role separation unclear; pick one flow for V1 and add the other later	Medium
Calendar / Deadline Reminders	Partial implementation; notification system not built	Medium

Table 8: Deferred features with rationale

4.3 CUT: Remove Entirely from Codebase

These features should be removed from the codebase entirely. They are either orphaned code that no user will ever interact with, features that add complexity without proportional value, or features that are so far from completion that they represent sunk cost rather than future value. Removing them reduces the attack surface, simplifies the data model, and eliminates dead code that confuses developers and obscures the real architecture.

Feature / Code	Reason for Removal	Impact of Removal
PropertyTrackingView	Orphaned component; no route points to it; no user can access it	Positive: removes dead code and confusion
property.rentPaymentHistory	Duplicate of ledger_entries; creates data divergence; no reconciliation	Positive: single source of truth for payments
Legal Research Integration	Not started; no API partner identified; high complexity	None: was never built

Court Filing Integration	Not started; requires Nigerian court system API access which does not exist	None: was never built
Custom Workflow Automation	Partial implementation adds complexity without usable output; no user has requested this	Positive: simplifies DataProvider
Excess Context Providers	12 providers can be consolidated to 4-5 without losing functionality	Positive: faster renders, easier debugging

Table 9: Features and code to remove entirely

4.4 Recommended V1 Scope

After applying the KEEP, DEFER, and CUT recommendations, the V1 scope of PracticePro is dramatically simpler while still delivering a complete, valuable product for the Nigerian market. The V1 product is a focused legal practice management tool with core property management capabilities, specifically the rent collection and tracking features that are most urgently needed by Nigerian property lawyers and managers. All AI features, advanced property features, and speculative integrations are removed from V1 scope.

Aspect	Before Simplification	After Simplification	Reduction
Total features	35	15	57% fewer
Working features	10 (29%)	15 (100% target)	All must work
Context providers	12	4-5	58% fewer
Payment systems	2 (unsynchronized)	1 (ledger_entries)	50% fewer
API key exposure	2 exposed keys	0 (server-side)	100% resolved
Auth bypass	Present	Fixed	100% resolved
Audit logging	None	Basic (login, data, deletes)	NBA minimum

Table 10: V1 scope reduction summary

5. Audit Framework Mapping to PracticePro

The comprehensive 10-area audit framework for legal and proptech SaaS applications provides a structured approach to quality assurance. This section maps each area of the framework specifically to PracticePro's known issues, stack (React + Convex), and target market (Nigeria), identifying which items from the framework are already known issues, which are new gaps that need investigation, and which are not applicable to the current stage of development.

5.1 Performance

The framework targets Core Web Vitals of LCP under 2.5 seconds, FID under 100ms, and CLS under 0.1. PracticePro's 12-provider context hierarchy is the primary performance risk. Each context provider adds a re-render cost when its value changes, and with twelve providers, a single state change can trigger cascading re-renders across the entire component tree. The Mega-Query refactor may have improved data fetching performance, but it may also have introduced waterfalls or redundant queries that are not yet measured. The bundle size is currently unknown; the framework targets an initial bundle under 200KB gzipped and a main bundle under 500KB gzipped. PracticePro includes the pdfkit library for receipt generation, which alone can add 200-300KB to the bundle. Running a Lighthouse audit on the current build should be the first action, followed by a bundle analysis to identify the largest contributors.

5.2 Security

Security is the area where PracticePro has the most critical gaps. The framework recommends npm audit, Snyk, OWASP ZAP, and OWASP Top 10 compliance. PracticePro has confirmed violations of at least three OWASP Top 10 categories: A01 (Broken Access Control) via the withFirmAuth bypass, A05 (Security Misconfiguration) via exposed API keys, and A01 again via IDOR (any user can access any firm's data). These are not theoretical risks; they are confirmed, reproducible vulnerabilities. Running OWASP ZAP against the deployed application would likely reveal additional vulnerabilities, but the three known issues are severe enough that they must be fixed before any external penetration testing. The exposed API keys also violate the framework's recommendation for secrets management, and the absence of rate limiting on Convex functions means that an attacker could potentially brute-force access to firm data without triggering any alerts.

5.3 Accessibility

The framework targets WCAG 2.1 AA compliance, recommending tools like axe DevTools and Pa11y. PracticePro has not been tested for accessibility, which is a significant gap in the Nigerian market where web accessibility is increasingly mandated by the Discrimination Against Persons with Disabilities (Prohibition) Act 2018. The modal context discard issue may also cause accessibility problems, as screen readers may not properly announce modal content when context is lost. Running an automated accessibility audit with axe should be a priority, but it should be done after the V1 simplification to avoid wasting effort on features that will be cut.

5.4 Code Quality

The framework recommends ESLint, SonarQube, depcheck, and license compliance checks. PracticePro likely has significant unused dependencies given the features being cut (PropertyTrackingView, AI integrations). Running depcheck after the simplification will identify orphaned dependencies. SonarQube analysis would quantify the technical debt, but the manual audit already identified the key issues: the dual payment system, the broken auth wrapper, and the excessive context providers. The priority should be fixing these known issues rather than adding another analysis tool.

5.5 Data Privacy & Compliance

This is a critical area for PracticePro. The framework emphasizes GDPR, but the Nigerian equivalent is the Nigeria Data Protection Regulation (NDPR) and the newer Nigeria Data Protection Act (NDPA) 2023. These require data minimization, purpose limitation, consent management, and the right to erasure. PracticePro currently has no consent management system, no data export functionality (required for the right to data portability), and no deletion mechanism that properly cascades through related records. The 7-year data retention requirement from the NBA adds complexity: the system must retain certain records for 7 years while also supporting the user's right to erasure under NDPA. This tension must be resolved through a clear data retention policy that distinguishes between client data (subject to erasure requests) and professional records (subject to retention requirements).

5.6 API Audits

The framework recommends rate limiting, authentication verification, and input validation for all API endpoints. In the Convex context, every exported mutation and query is an API endpoint. The `withFirmAuth` bypass means that authentication verification is effectively absent on all protected endpoints. Input validation in Convex uses the built-in argument validation system (`v.string()`, `v.number()`, etc.), which provides basic type safety but does not validate business rules (e.g., that a payment amount is positive, that a date is not in the future, that a `firmId` matches the authenticated user's firm). Rate limiting is not natively supported by Convex and would need to be implemented as a custom middleware or through Convex action wrappers.

5.7 Infrastructure

The framework targets SSL/TLS A+ rating and regular backups. Convex handles SSL/TLS automatically, so this is not a concern. However, Convex's backup capabilities are limited. The framework recommends daily backups with point-in-time recovery, which Convex supports through its built-in snapshot system. The concern is that Convex snapshots are internal to Convex and cannot be exported to a separate storage provider. For a legal application, having a backup that is controlled by a third-party service may not meet NBA requirements for data custody. This should be investigated with Convex support.

5.8 UX

The framework recommends Hotjar for user behavior analytics and a maximum of 3 clicks for critical flows. PracticePro's current UX suffers from feature overload: users see both legal and property management features regardless of their role. The simplification roadmap addresses this by removing non-essential features, but the information architecture also needs attention. The 3-click rule should be applied to the V1 features: creating a matter, collecting rent, and generating a receipt should all be achievable in 3 clicks or fewer from the dashboard.

5.9 Database

The framework identifies orphaned records, `firmId` indexes, and cascade deletes as critical database concerns. All three apply directly to PracticePro. The dual payment system creates orphaned records when one system is updated but not the other. The absence of `firmId` indexes on some Convex tables means that

queries filtering by firm must perform full table scans, which will degrade performance as the database grows. The nuclear delete concern (deleting a firm should cascade to all related data) is not implemented and could leave orphaned records across the database. After the simplification, the database schema should be audited to ensure every table has a firmId index and that delete operations properly cascade.

5.10 Compliance Audit Logs

The framework provides a Convex audit log implementation that serves as a template. PracticePro must implement audit logging for all data access and mutation events, not just for NBA compliance but also for operational debugging. The audit log schema should include: userId, action (using a standardized vocabulary like MATTER_VIEWED, DOCUMENT_DOWNLOADED, PAYMENT_RECORDED), resourceType, resourceId, timestamp, and optional metadata (ipAddress, userAgent). For V1, logging should cover at minimum: authentication events (login, logout, failed attempts), data access events (viewing a matter, downloading a document), and mutation events (creating, updating, or deleting any record). The implementation should use a dedicated Convex table with a TTL or archival policy to manage storage growth over time.

6. Priority Action Plan

This section provides a sequenced action plan that accounts for dependencies between tasks. The plan is organized into four phases, each building on the completion of the previous phase. The total estimated effort assumes a single full-stack developer working on the Ati Gravity platform, with Convex backend and React frontend.

6.1 Phase 1: Security Emergency (Week 1-2)

Task	Description	Priority	Est. Days
Fix withFirmAuth	Add getUserIdentity() call to withFirmAuth; verify identity on every protected function	P0	2
Move API keys server-side	Create Convex actions for Gemini API calls; remove keys from client bundle	P0	2
Remove PropertyTrackingView	Delete orphaned component and all imports; verify no runtime errors	P1	0.5
Consolidate payment systems	Remove property.rentPaymentHistory; migrate data to ledger_entries; update all queries	P0	3
Run npm audit + fix critical	Address all critical and high-severity dependency vulnerabilities	P1	1

Table 11: Phase 1 action items

6.2 Phase 2: Core Fixes (Week 3-4)

Task	Description	Priority	Est. Days
Fix CollectRentModal	Resolve modal context discard; ensure tenant and property data available in modal	P0	2
Fix ReceiptDetailView	Fix receipt generation at unit level; ensure pdfkit works correctly	P0	2
Fix Add Payment Flow	Connect to consolidated ledger_entries; unit-level context	P0	1.5
Implement basic audit logging	Convex auditLog table + logAction mutation for auth, data access, deletes	P1	2
Separate DataProviders	Split DataProvider.tsx into MatterDataProvider and PropertyDataProvider	P1	2

Table 12: Phase 2 action items

6.3 Phase 3: Simplification (Week 5-6)

Task	Description	Priority	Est. Days
Flatten context providers	Consolidate 12 providers to 4-5; remove redundant contexts	P1	3
Remove deferred features	Comment out or remove routes/components for AloaChat, valuations, bulk edit, maintenance, activity	P1	2
Simplify tier enforcement	Remove Revenue Monitor from Free/Pro tiers; add Custom/Enterprise gate	P2	1
Add firmId indexes	Audit all Convex tables; add firmId indexes where missing	P1	1
Run depcheck + clean	Remove unused dependencies after feature cuts	P2	1

Table 13: Phase 3 action items

6.4 Phase 4: Quality & Launch Prep (Week 7-8)

Task	Description	Priorit y	Est. Days
Lighthouse audit	Run full Lighthouse audit; fix Core Web Vitals to meet framework targets	P1	2
Accessibility scan	Run axe DevTools on all V1 pages; fix critical violations	P2	2
E2E testing	Write Playwright/Cypress tests for all V1 critical paths	P1	3
Nigerian tax validation	Test withholding tax + VAT calculations with real Nigerian scenarios	P1	1
NBA compliance review	Review audit logs, retention policy, and confidentiality controls against NBA standards	P1	1

Table 14: Phase 4 action items

7. The Case for Radical Simplicity

PracticePro is at an inflection point. The current trajectory, adding features faster than they can be stabilized, will lead to a product that is impressive in its ambition but unusable in its execution. The Nigerian legal market does not need a Swiss-army-knife application; it needs a reliable, simple tool that solves the most painful problems first. Law firms are conservative adopters of technology. They will not tolerate bugs, security vulnerabilities, or confusing interfaces. The first product that earns their trust will have an enormous competitive advantage.

The simplification roadmap reduces PracticePro from 35 features to 15, eliminates the two most dangerous security vulnerabilities, consolidates the dual payment system into a single source of truth, and flattens the context provider tree from 12 to 4-5 providers. The result is a product that can be shipped in 8 weeks with confidence that every feature works, every data access is authenticated, and every financial record is consistent. This is not a lesser product; it is a better product, because every feature it includes works reliably and every feature it excludes was not yet ready to work reliably.

The features that are deferred or cut are not abandoned; they are sequenced. AloaChat, property valuations, maintenance tracking, and the revenue monitor will all return in V2, but they will return on a foundation of security, stability, and user trust. The goal of V1 is not to be feature-complete; it is to be reliability-complete. A product that does fewer things but does them perfectly will always outperform a product that does everything but breaks unpredictably. This is the radical simplicity that PracticePro needs to succeed in the Nigerian market.